

Assisted Labs

For more details, please visit <https://assistedlabs.com/>

UNDERSTANDING OPENSIFT CONFIGMAP: WHAT IT IS AND HOW TO USE IT

In cloud-native development, separating configuration from code improves flexibility and maintainability. OpenShift uses **ConfigMap** to store non-sensitive data like environment variables or config files outside the application image.

WHAT IS A CONFIGMAP?

A **ConfigMap** is a key-value pair object in OpenShift/Kubernetes to externalize configuration.

Common use cases:

- Environment variables (e.g., `APP_MODE=production`)
- Config files (YAML, JSON)
- External URLs or service settings

Use **Secrets** for sensitive data like passwords or tokens.

WHY AND WHERE TO USE CONFIGMAPS

Why:

- Decouples config from code
- Enables reuse across pods
- Supports dynamic config updates
- Easy rollback with Git
- Environment-specific configs

Where:

- Pods (runtime variables)
- Deployments (dynamic values)
- Volumes (mount full files)
- CI/CD pipelines (per environment)
- CronJobs/StatefulSets (scheduled workloads)

CREATING AND USING CONFIGMAPS

CREATE

Assisted Labs

For more details, please visit <https://assistedlabs.com/>

```
oc create configmap my-config --from-literal=APP_MODE=production
oc create configmap my-config --from-file=app.properties
```

MANAGE

```
oc get configmap my-config
oc describe configmap my-config
oc edit configmap my-config
```

RECREATE

```
oc delete configmap my-config
oc create configmap my-config --from-literal=KEY=value
```

INJECT INTO APP (YAML)

```
envFrom:
  - configMapRef:
      name: my-config

volumes:
  - name: config-volume
    configMap:
      name: my-config
volumeMounts:
  - name: config-volume
    mountPath: /etc/config
```

WHEN AND HOW TO USE

Use ConfigMaps when:

- Config varies per environment
- You need to avoid hardcoding
- Managing microservices with shared settings
- The data is not confidential

EXAMPLE YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
```

Assisted Labs

For more details, please visit <https://assistedlabs.com/>

```
APP_MODE: "production"
DB_HOST: "mysql-service"
DB_PORT: "3306"
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sample-app
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: app-container
          image: myapp:latest
          envFrom:
            - configMapRef:
                name: app-config
```

CONSIDERATIONS & TOOLING

LIMITATIONS

- Not encrypted
- No live updates unless app supports reload
- 1MB max size

RELOAD OPTIONS

- Restart pods manually: `oc delete pod <pod-name>`
- Sidecars: Reloader, configmap-reload
- Framework watchers: Spring Cloud, Node.js fs.watch

TOOL INTEGRATION

- **Helm:** Templated config
- **Argo CD:** Git-tracked updates
- **Jenkins:** CLI automation
- **Kustomize:** Environment overlays

SECURITY AND BEST PRACTICES

- Don't store secrets in ConfigMaps

Assisted Labs

For more details, please visit <https://assistedlabs.com/>

- Use RBAC to control access
- Enable audit logging
- Use versioned ConfigMaps for rollback
- Automate via CI/CD pipelines
- Test config in staging first

SUMMARY TABLE

Category	Details
What	Store non-sensitive config (key-value format)
Why	Decouple config from code
Where	Pods, deployments, CI/CD, volumes
How	CLI/YAML, env vars, or mounted files
When	When config must be updated independently
Benefits	Flexible, reusable, CI/CD & Git friendly

CONCLUSION

OpenShift ConfigMaps simplify app configuration by keeping settings separate from code. They're ideal for modern, modular, and environment-specific deployments.

By using ConfigMaps properly, you ensure faster, cleaner, and more reliable application management.

For more details, please visit <https://assistedcloud.com>

Assisted Labs

For more details, please visit <https://assistedlabs.com/>